SCCM7450_Week1_Practical

September 2026

1 Python Basics, NumPy and Git



Python was first created in 1991 by Guido van Rossum. He defined the aims of Python to be:

- An easy and intuitive language just as powerful as major competitors
- Open source, so that anyone can contribute to its development
- Code that is as understandable as plain English
- Suitability for everyday tasks, allowing for short development times

Python is a great general-purpose programming language on its own, but with the help of a few popular libraries (e.g., numpy, scipy, pandas, sklearn, matplotlib), it becomes a powerful environment for scientific computing.

Therefore, we have chosen Python as the programming language for SCCM7450 Introduction to AI and Data Science. This lab will serve as a quick crash course in Python basics to provide you with a head start.

In this tutorial, we will cover:

- **Python Basics:** variables, comments, basic data types, containers, conditionals, functions and classes
- **NumPy:** arrays, array indexing, data types, array math and broadcasting

We will also provide some references and tutorials to help you refresh your knowledge or become familiar with **Git version control**.

1.1 Getting Started

1.1.1 Python File Formats

In Python, you will often see two common types of files:

- **`.py` files:** These are plain Python scripts. They contain only code, written line by line, and run from the command line or an editor. If you want to write a program that can be reused, or run as a standalone application, `.py` files are usually the best choice.
- **`.ipynb` files:** These are Jupyter Notebook files. They are especially popular in data science and machine learning. Unlike a `.py` file, a notebook can combine Python code, its output (like tables or charts), and rich text written in Markdown.

You are free to select one file format or use both. The IDE recommended below under programming environments supports both file formats; however, the cloud-based environment is mainly designed for notebooks.

(IDE stands for Integrated Development Environment. It is a software application that provides all the main tools you need to write, test, and debug code in one place.)

1.1.2 Python Versions

As of January 1, 2020, Python has officially dropped support for `Python2`. We will be using `Python 3.12` for all the practical sessions in this module.

You can check your Python version at the command line by running `python --version`.

However, if you run this on a Jupyter Notebook, you need to add an `!` before the command to indicate that it is a shell command, not Python code.

[ ]:
```
!python --version
```

1.1.3 Programming Environments

We recommend two commonly used programming environments for you to use during this module.

1. Visual Studio Code (VS Code)
2. Google Colab

If you are already familiar with any other programming environment, you are very welcome to continue using it.

Visual Studio Code (VS Code) VS Code is a free code editor that you can install on your computer (It has already been installed in the lab machines). It works for many programming languages, including Python, and comes with helpful features, like debugging tools.

Getting Started with Python in VS Code: This tutorial will guide you with the initial setup in VS Code.

Please note that lab machines already have Python virtual environments created - `sccm-7450` includes all the libraries/packages you will require for this module installed. You can also create your own Python virtual environments and install packages via pip; however, these will be ephemeral and will need re-creating at each login.

If you prefer having Jupyter Notebooks instead of plain Python scripts, Jupyter Notebooks in VS Code will provide you with a good head start.

Google Colab Google Colab is a cloud-based platform that lets you run Python code in your browser. It is mainly designed for Jupyter Notebooks. Colab environments come with some ready-to-go configurations, especially designed for data science and machine learning. It also provides free access to GPUs for faster computation. However, you will need a Google account to access Colab.

Using the following command, you can see all the libraries available in your Colab environment.

```
!pip freeze
```

You will only need to install a library if it is not already available or if you prefer another version of it.

Have you decided which programming environment to use?

Once you have it ready, you can try the following exercises to learn more about Python.

1.2 The Zen of Python

The Zen of Python is a collection of guiding principles for writing Python code, created by Tim Peters.

You can actually see it in Python by running:

[ ]:
```
import this
```

It will print out 19 short aphorisms (like mini-rules). They are not strict rules, but more of a philosophy to encourage code that is clean, simple, and readable, aligning with Python's design philosophy.

There is also an official style guide, called PEP 8, that provides concrete rules and recommendations for writing Python code. It guides you on how to write Python code clearly and consistently, making it easier for others (and your future self) to read and maintain.

1.3 Variables

Variables are containers for storing data values.

A variable is created the moment you first assign a value to it.

[ ]:
```
x = 3
```

You must have noticed that we do not define a type when we create a variable. Therefore, in Python, you cannot have variables without an assignment, as shown below.

[ ]:
```
var # NameError: 'var' is not defined
```

Python variables infer their type when they are assigned. These types of languages are known as **dynamic typed languages**.

Variables can therefore also change type during the duration of your program.

[ ]:
```
x = 3
x = "Hello"
x = True
```

Could you guess the value of x after running the above code snippet before running it?

1.3.1 Naming Variables

A variable can have a short name (like x, we used above) or a more descriptive name (like age and total_volume). It is always recommended to use a descriptive or meaningful name to improve clarity and avoid confusion in your code.

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can can contain letters, digits, and underscores (_)
- Variable names are case-sensitive (age, Age and AGE are three different variables)

Python encourages **snake_case** for most identifiers (i.e., variable names and function names) to keep code consistent and readable. Snake case is a naming convention where words are written in lowercase and separated by underscores, as below.

[ ]:
```
first_name = "John"
second_name = "Doe"
```

Python also supports **multiple assignment** to assign several distinct values to multiple variables and **single assignment** to assign the same value to multiple variables, as below.

[ ]:
```
# multiple assignment
x, y, z = "red", "green", "blue"
print(x, y, z)

# single assignment
a = b = c = "red"
print(a, b, c)
```

1.3.2 Casting

If you want to specify or change the data type of a variable, it can be done with casting.

Below are some common constructor functions used for casting:

- int() - constructs an integer number from an integer literal, a float literal, or a string literal (providing the string represents a whole number)
- float() - constructs a float number from an integer literal, a float literal or a string literal (providing the string represents a float or an integer)
- str() - constructs a string from a wide variety of data types, including strings, integer literals and float literals

[ ]:
```
x = '100'
print(int(x))
```

However, casting will fail if the value cannot be reasonably converted to the target type, as shown below.

[ ]:
```
x = 'hello'
print(int(x))   # ValueError: 'hello' is an invalid literal for int
```

1.4 Comments

Comments are parts of your code that are ignored by the interpreter. They are written for humans rather than computers. Comments can help explain what the code does, why a particular approach was taken, or leave reminders for future work.

There is one dedicated type of comment in Python:

1. Single-line comments: Use the `#` symbol.

For longer explanations spanning multiple lines, you can use triple-quoted strings (`"""` or `'''`). These are technically string literals rather than comments, but they are sometimes used in a similar way. Triple-quoted strings are also commonly used for docstrings, which document functions, classes, and modules.

[ ]:
```
# This is a single-line comment
print("Hello, World!")
```

[ ]:
```
"""
This is a comment
written in
more than just one line
"""
print("Hello, World!")
```

Comments are important for several reasons: - Readability: Makes code easier for others (and yourself) to understand

- Maintenance: Helps explain why certain decisions were made, which is useful when updating code later
- Debugging: You can temporarily "comment out" code to test or debug without deleting it
- Collaboration: Teams can understand each other's code quickly through clear comments

Always keep comments clear and concise, and avoid just stating the obvious.

1.5 Basic Data Types

1.5.1 Numbers

Integers and floats work as you would expect from other languages:

[ ]:
```
x = 3
print(x, type(x))
```

[ ]:
```
print(x + 1)   # Addition
print(x - 1)   # Subtraction
print(x * 2)   # Multiplication
print(x ** 2)  # Exponentiation
```

[ ]:
```
x += 1
print(x)
x *= 2
print(x)
```

[ ]:
```
y = 2.5
print(type(y))
print(y, y + 1, y * 2, y ** 2)
```

Note that, unlike many languages, Python does not have unary increment (`x++`) or decrement (`x--`) operators. To increment or decrement a variable, use `+=` or `-=`, for example, `x += 1`.

Python has built-in numeric types for integers, floating-point numbers, and complex numbers. You can find more details in the Python documentation

1.5.2 Booleans

Python implements all of the usual operators for Boolean logic, but uses English words rather than symbols (&&, ||, etc.):

[ ]:
```
t, f = True, False
print(type(t))
```

[ ]:
```
print(t and f) # Logical AND;
print(t or f)  # Logical OR;
print(not t)   # Logical NOT;
print(t != f)  # Logical XOR;
```

Truthiness in Python Python extends the concept of true and false beyond just the Boolean values True and False, allowing numbers, sequences, collections, and other objects to be evaluated in a Boolean context.

In Python, many things are true:

- Numeric values other than zero
- Non-empty data structures
- Strings of length >0
- True

Anything that is not "truthy" is "falsy".

[ ]:
```
# when x is a number
x = 0
if x:
    print(x, "is True")

x = 1
if x:
    print(x, "is True")
```

[ ]:
```
# when x is a String
x = "Hello"
if x:
    print(x, "is True")

x = ""
if x:
    print(x, "is True")
```

(You will see more details about strings and ifs in a moment.)

Why does Python do this?

- Simplifies code: You do not have to write `if x!=0:` — just `if x:`
- Makes conditions more readable and Pythonic.
- Helps avoid unnecessary comparisons, keeping your code clean and concise.

1.5.3 Strings

[ ]:
```
hello = 'hello'   # String literals can use single quotes
world = "world"   # or double quotes; it does not matter
print(hello, len(hello))
```

[ ]:
```
hw = hello + ' ' + world  # String concatenation
print(hw)
```

[ ]:
```
hw12 = '{} {} {}'.format(hello, world, 12)  # string formatting
print(hw12)

hw12 = f'{hello} {world} 12' # easy to read and more concise formatting␣
 ↪(introduced in Python 3.6.)
print(hw12)
```

String objects have a bunch of useful methods; for example:

[ ]:
```
s = "hello"
print(s.capitalize())  # Capitalize a string
print(s.upper())       # Convert a string to uppercase; prints "HELLO"
```

```
print(s.replace('l', '(ell)'))  # Replace all instances of one substring with another
print('  world '.strip())  # Strip leading and trailing whitespace
```

You can find a list of all string methods in the documentation.

Task 1: Define two strings, s and r, and assign them 'I am learning python' and 'learning' respectively. Write the code to find whether s contains r or not. In case r is contained, your program should print the index.

1.6 Containers

Python includes several built-in container types: lists, tuples, sets and dictionaries.

Each one is designed to solve different problems efficiently. They offer different behaviours, capabilities, and performance characteristics.

- **Lists:** Ordered, mutable, allow duplicate elements. Useful for sequences of data.
- **Tuples:** Ordered, immutable, allow duplicate elements. Effectively immutable lists.
- **Sets:** Unordered, mutable, contains unique elements. Useful for eliminating duplicates from, e.g., lists.
- **Dictionaries:** Ordered (Python 3.7+), mutable, mapped by key-value pairs. Useful for fast lookup.

1.6.1 Lists

A list is the Python equivalent of an array, but is resizeable and can contain elements of different types:

[ ]:
```
xs = [3, 1, 2]  # Create a list
print(xs)
```

[ ]:
```
print(xs[0])    # Access the first element of the list by index
print(xs[-1])     # Negative indices count from the end of the list; prints "2"
```

[ ]:
```
xs[2] = 'hello'   # Lists can contain elements of different types
print(xs)
```

[ ]:
```
xs.append('world')    # Add a new element to the end of the list
print(xs)
```

[ ]:
```
x = xs.pop()     # Remove and return the last element of the list
print(x, xs)
```

As usual, you can find all the details about lists in the documentation.

Task 2: Write the code to merge two lists x and y while maintaining the order, and the resultant list should be stored in a new list z, where x = [1,2,3] and y=['a','b','c'] and z should be [1,2,3,'a','b','c'].

Slicing In addition to accessing list elements one at a time by index, Python provides concise syntax to access sublists; this is known as slicing:

[ ]:
```
nums = list(range(5))    # range() creates a sequence of integers; list()␣
 ↪converts it to a list
print(nums)          # Prints "[0, 1, 2, 3, 4]"
print(nums[2:4])     # Get a slice from index 2 to 4 (exclusive); prints "[2, 3]"
print(nums[2:])      # Get a slice from index 2 to the end; prints "[2, 3, 4]"
print(nums[:2])      # Get a slice from the start to index 2 (exclusive); prints␣
 ↪"[0, 1]"
print(nums[:])       # Get a slice of the whole list; prints "[0, 1, 2, 3, 4]"
print(nums[:-1])     # Slice indices can be negative; prints "[0, 1, 2, 3]"
nums[2:4] = [8, 9]   # Assign a new sublist to a slice
print(nums)          # Prints "[0, 1, 8, 9, 4]"
```

Loops You can loop over the elements of a list like this:

[ ]:
```
animals = ['cat', 'dog', 'monkey']
for animal in animals:
    print(animal)
```

If you want access to the index of each element within the body of a loop, use the built-in enumerate function:

[ ]:
```
animals = ['cat', 'dog', 'monkey']
for idx, animal in enumerate(animals):
    print(f'#{idx}: {animal}')
```

List comprehensions When programming, we often want to transform a data element into another. As a simple example, consider the following code that computes square numbers:

[ ]:
```
nums = [0, 1, 2, 3, 4]
squares = []
for x in nums:
    squares.append(x ** 2)
print(squares)
```

You can make this code simpler using a list comprehension:

[ ]:
```
nums = [0, 1, 2, 3, 4]
squares = [x ** 2 for x in nums]
print(squares)
```

List comprehensions can also contain conditions:

[ ]:
```
nums = [0, 1, 2, 3, 4]
even_squares = [x ** 2 for x in nums if x % 2 == 0]
print(even_squares)
```

1.6.2 Tuples

A tuple is an (immutable) ordered list of values.

[ ]:
```
t = (1, 2, 3)   # Tuples are created using round brackets - ()
```

[ ]:
```
print(t[0]) # Access the first element by index
print(t[1:2])   # Tuples can also be sliced similar to lists
```

However, unlike lists, tuples cannot be updated. The following code will throw an error, as mutability is not permitted in tuples.

[ ]:
```
t[0] = 4    # TypeError: Tuples do not support item assignment
```

A tuple is in many ways similar to a list; one of the most important differences is that tuples can be used as keys in dictionaries and as elements of sets, while lists cannot. Here is a trivial example:

[ ]:
```
d = {(x, x + 1): x for x in range(10)}  # Create a dictionary with tuple keys
print(d)
t = (5, 6)       # Create a tuple
print(type(t))
print(d[t])
print(d[(1, 2)])
```

1.6.3 Sets

A set is an unordered collection of distinct elements.

[ ]:
```
animals = {'dog', 'cat'} # Sets are created using curly brackets - {}
print(animals)
```

[ ]:
```
print(type({}))           # But {} on its own is an empty DICT
print(type(set()))        # Use set() for an empty set
```

[ ]:
```
print('cat' in animals)   # Check if an element is in a set; prints "True"
print('fish' in animals)  # prints "False"
```

[ ]:
```
animals.add('fish')      # Add an element to a set
print(animals)
print('fish' in animals)
print(len(animals))       # Number of elements in a set;
```

[ ]:
```
animals.add('cat')       # Adding an element that is already in the set does␣
 ↪nothing
print(animals)
animals.remove('cat')    # Remove an element from a set
print(animals)
```

Iterating over a set has the same syntax as iterating over a list; however, since sets are unordered, you cannot make assumptions about the order in which you visit the elements of the set:

```
animals = {'cat', 'dog', 'fish'}
for idx, animal in enumerate(animals):
    print(f'#{idx}: {animal}')
```

Like lists, we can easily construct sets using set comprehensions:

```
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print({int(x/2) for x in nums})
```

Task 3: Think what will happen if we run this code list(set([1,2,3,3,2,1])). Verify your answer by running it.

1.6.4 Dictionaries

A dictionary stores (key, value) pairs, similar to a Map in Java or an object in JavaScript. You can use it like this:

```
# Dictionaries are created using curly brackets - {},
# but they have key:value pairs per element
d = {'cat': 'cute', 'dog': 'furry'}
print(d)
print(d['cat'])       # Get an entry from a dictionary using key; prints "cute"
print('cat' in d)     # Check if a dictionary has a given key; prints "True"
```

```
d['fish'] = 'wet'    # Set an entry in a dictionary
print(d)
print(d['fish'])      # Prints "wet"
```

```
print(d['monkey'])  # KeyError: 'monkey' not a key of d
```

To avoid the above error, you can either check if a dictionary has the given key before accessing it or access an element with a default value.

```
print(d.get('monkey', 'N/A'))  # Get an element with a default; prints "N/A"
print(d.get('fish', 'N/A'))    # Get an element with a default; prints "wet"
```

```
del d['fish']         # Remove an element from a dictionary
print(d.get('fish', 'N/A')) # "fish" is no longer a key; prints "N/A"
```

You can find all you need to know about dictionaries in the documentation.

You can iterate over dictionaries using keys or key-value pairs.

```
d = {'person': 2, 'cat': 4, 'spider': 8}
# Iterate using keys
for animal in d.keys():
    print(f'A {animal} has {d[animal]} legs')
```

```
# Iterate using key-value pairs
for animal, legs in d.items():
```

```
    print(f'A {animal} has {legs} legs')
```

Dictionaries can also be easily constructed using dictionary comprehension, similar to list comprehension.

[ ]:
```
nums = [0, 1, 2, 3, 4]
even_num_to_square = {x: x ** 2 for x in nums if x % 2 == 0}
print(even_num_to_square)
```

1.7 Conditionals

Conditionals in Python allow your program to make decisions and execute different code depending on whether a condition is True or False.

We have used a few conditionals so far in the content. For example, `if x % 2 == 0` in the above dictionary comprehension example, ask the code to calculate the square of x, if the remainder when x is divided by 2 equals zero.

Let us take a deep dive into conditionals.

Python supports the usual logical conditions (or comparisons) from mathematics:

- Equals: `a == b`
- Not Equals: `a != b`
- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`

Conditionals are commonly used in if statements and loops.

If Statements

[ ]:
```
age = 18
if age >= 18:
    print("You are an adult.")
```

[ ]:
```
# else provides an alternative block to run when the condition is False.
age = 16
if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

Python also allows inline ifs to make the code simpler, as shown below.

[ ]:
```
age = 16
print("Adult" if age >= 18 else "Minor")
```

[ ]:
```
# Or we can have elif to check multiple conditions sequentially
score = 85
if score >= 90:
    print("Grade: A")
elif score >= 75:
    print("Grade: B")
elif score >= 60:
    print("Grade: C")
else:
    print("Grade: F")
```

Note that Python relies on **indentation/ semantic whitespace** (whitespace at the beginning of a line) to define a code block or scope in the code. Other programming languages, like Java, often use curly brackets for this purpose.

You must have the indentations right to convey the right meaning. See the difference in outputs generated by the following code snippets. One missed indentation could completely alter the behaviour of your code.

[ ]:
```
age = 16
if age >= 18:
    age_category = 'adult'
    print("You are an adult.")
```

[ ]:
```
age = 16
if age >= 18:
    age_category = 'adult'
print("You are an adult.")
```

Match Statement Python introduced the match statement starting with Python 3.10. It is similar to a switch in other languages. You can use the match statement instead of writing many if-else statements.

[ ]:
```
x = 7

match x:
    case 1:
        print("One")
    case 2:
        print("Two")
    case 3:
        print("Three")
    case _:
        print("Something else")
```

_ is the wildcard pattern. It defines the default case.

Also, `match` is considered more powerful. You can combine several literals in a single pattern using | ("or") and even unpack structures using match. More information about the match statement is

available in the documentation.

Loops: while While loops follow a similar syntax to if statements, but instead of running the block only once, they repeatedly execute it as long as the condition remains True. The loop stops when the condition becomes False.

[ ]:
```
i = 1
while i < 6:
  print(i)
  i += 1
```

Note that if we did not add the last line to increment i, the loop would continue forever.

We can stop the loop when a condition is met using the **break statement**.

[ ]:
```
i = 1
while i < 6:
  print(i)
  if i == 2:
      break
  i += 1
```

Loops: for For loops are commonly used to iterate over sequences (e.g., a list, tuple, set, dictionary or string).

The following example shows how to iterate through a list using a for loop, as you have seen earlier in this tutorial.

[ ]:
```
animals = ['cat', 'dog', 'monkey']
for animal in animals:
    print(animal)
```

We can similarly iterate through a string. Strings contain a sequence of characters.

[ ]:
```
for x in "banana":
  print(x)
```

The `range()` function can be used to generate a sequence of numbers.

We can use `range()` with different setups. Note that the end is exclusive, the start defaults to 0, and the step defaults to 1.

- range(end)
- range(start, end)
- range(start, end, step)

[ ]:
```
print('\n range(5)')
for i in range(5):  # Iterate through 0, 1, 2, 3, 4
    print(i)
```

```
print('\n range(2, 5)')
for i in range(2, 5):   # Iterate through 2, 3, 4
    print(i)

print('\n range(1, 5, 2)')
for i in range(1, 5, 2):   # Iterate through 1, 3
    print(i)
```

1.8 Functions

Python functions are defined using the `def` keyword. They do not require a return type or parameter types to be specified. Return values are also optional.

[ ]:
```
def sign(x):
    if x > 0:
        return 'positive'
    elif x < 0:
        return 'negative'
    else:
        return 'zero'

for x in [-1, 0, 1]:
    print(sign(x))
```

Type hints can be added optionally to indicate the expected types of parameters and return values.

[ ]:
```
def sign(x: int) -> str:
    if x > 0:
        return 'positive'
    elif x < 0:
        return 'negative'
    else:
        return 'zero'

for x in [-1, 0, 1]:
    print(sign(x))
```

Function parameters are comma-separated and can be optional. Default values may be optionally assigned to parameters when defining the function.

[ ]:
```
def hello(name, loud=False):
    if loud:
        print(f'HELLO, {name.upper()}')
    else:
        print(f'Hello, {name}!')

hello('Bob')
```

```
hello('Fred', loud=True)
```

Task 4: Write a math function which takes three parameters, namely a, b and op, and returns the value as: value of a+b if op is '+', value of a-b if op is '-', value of a * b if op is '*' and -1 for any other op.

1.9 NumPy

NumPy is the core library for scientific computing (i.e., working with numbers, arrays and matrices) in Python. It provides a high-performance multidimensional array object, called ndarray, which allows you to store and manipulate large datasets efficiently. With NumPy, you can perform mathematical operations on entire arrays at once, making your code much faster and cleaner compared to using Python lists.

It also forms the foundation for many data science and machine learning libraries such as Pandas, SciPy, and Scikit-learn, making it one of the most important tools to learn after Python basics.

To use NumPy, we first need to import the numpy package:

If it is not available in you Python environment, you must first install it using the following command.

```
pip install numpy
```

[ ]:
```
import numpy as np
```

1.9.1 Arrays

A NumPy array is like a grid that holds values of the same type (all integers, all floats, etc.). Each value can be accessed using one or more indices that specify its position in the array.

The number of dimensions an array has determines how its values are organised. For example, a NumPy array containing a sequence of numbers is a one-dimensional (1-D) array, while an array arranged in rows and columns is a two-dimensional (2-D) array. The `shape` of an array is a tuple of numbers that shows how many elements it has along each dimension.

We can initialise NumPy arrays from nested Python lists and access elements using square brackets:

[ ]:
```
a = np.array([1, 2, 3])  # Create a 1-D array
print(type(a), a.shape, a[0], a[1], a[2])
a[0] = 5                 # Change an element of the array
print(a)
```

[ ]:
```
b = np.array([[1,2,3],[4,5,6]])   # Create a 2-D array
print(b)
```

[ ]:
```
print(b.shape)
print(b[0, 0], b[0, 1], b[1, 0])
```

NumPy also provides many functions to create arrays:

[ ]:
```
a = np.zeros((2,2)) # Create an array of all zeros
print(a)
```

[ ]:
```
b = np.ones((1,2))  # Create an array of all ones
print(b)
```

[ ]:
```
c = np.full((2,2), 7)   # Create a constant array
print(c)
```

[ ]:
```
d = np.eye(2)   # Create a 2x2 identity matrix
print(d)
```

[ ]:
```
e = np.random.random((2,2)) # Create an array filled with random values
print(e)
```

1.9.2 Array Indexing

Numpy offers several ways to index arrays.

Slicing Similar to Python lists, numpy arrays can be sliced. Since arrays may be multidimensional, you must specify a slice for each dimension of the array:

[ ]:
```
# Create the following 2-D array with shape (3, 4)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])

# Use slicing to pull out the subarray consisting of the first 2 rows
# and columns 1 and 2; b is the following array of shape (2, 2):
# [[2 3]
#  [6 7]]
b = a[:2, 1:3]
print(b)
```

A slice of an array is a view into the same data, so modifying it will modify the original array.

[ ]:
```
print(a[0, 1])
b[0, 0] = 77    # b[0, 0] is the same piece of data as a[0, 1]
print(a[0, 1])
```

You can also mix integer indexing with slice indexing. Since selecting a specific row or column with an integer removes that dimension, this approach will always result in an array with fewer dimensions than the original.

[ ]:
```
# Create the following 2-D array with shape (3, 4)
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
print(a)
```

We can access the middle row of the array using a mix of integer and slice indexing, as below. Note that the resulting array is 1D.

[ ]:
```
row_r1 = a[1, :]
print(row_r1, row_r1.shape)
```

However, the following two approaches will result in an array with the same number of dimensions as the original array:

[ ]:
```
row_r2 = a[1:2, :]  # 2-D view of the second row of a
row_r3 = a[[1], :]  # 2-D copy of the second row of a
print(row_r1, row_r1.shape)
print(row_r2, row_r2.shape)
print(row_r3, row_r3.shape)
```

Integer Array Indexing When you index NumPy arrays using slicing, the resulting array view will always be a subarray of the original array (it preserves the order and shape of the original data). In contrast, integer array indexing allows you to construct arbitrary arrays using the data from another array. Also, integer array indexing creates a new array, instead of returning a view. Here is an example:

[ ]:
```
a = np.array([[1,2], [3, 4], [5, 6]])
print(a, '\n')

# An example of integer array indexing.
# The returned array will have shape (3,)
print(a[[0, 1, 2], [0, 1, 0]], '\n')

# The above example of integer array indexing is equivalent to this:
print(np.array([a[0, 0], a[1, 1], a[2, 0]]), '\n')
```

[ ]:
```
b = a[[0, 1, 2], [0, 1, 0]]
print(b)
b[0] = 22
print(b)
print(a)
```

[ ]:
```
# When using integer array indexing, you can reuse the same
# element from the source array:
print(a[[0, 0], [1, 1]])

# Equivalent to the previous integer array indexing example
print(np.array([a[0, 1], a[0, 1]]))
```

One useful trick with integer array indexing is selecting or mutating one element from each row of a matrix:

[ ]:
```
# Create a new array from which we will select elements
a = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
print(a)
```

[ ]:
```
# Create an array of indices
b = np.array([0, 2, 0, 1])

# Select one element from each row of a using the indices in b
print(a[np.arange(4), b])  # Prints "[ 1  6  7 11]"
```

[ ]:
```
# Mutate one element from each row of a using the indices in b
a[np.arange(4), b] += 10
print(a)
```

Boolean Array Indexing Boolean array indexing lets you pick out arbitrary elements of an array. Frequently, this type of indexing is used to select the elements of an array that satisfy some condition. Here is an example:

[ ]:
```
a = np.array([[1,2], [3, 4], [5, 6]])

bool_idx = (a > 2)  # Find the elements of a that are bigger than 2;
                    # this returns a numpy array of Booleans of the same
                    # shape as a, where each slot of bool_idx tells
                    # whether that element of a is > 2.

print(bool_idx)
```

[ ]:
```
# We use boolean array indexing to construct a 1-D array
# consisting of the elements of a corresponding to the True values
# of bool_idx
print(a[bool_idx])

# We can do all of the above in a single concise statement:
print(a[a > 2])
```

You can find more information on NumPy array indexing by referring to the documentation.

1.9.3 Data Types

Every NumPy array is a grid of elements of the same type.

NumPy provides a large set of numeric datatypes that you can use to construct arrays. It tries to guess a datatype when you create an array, but functions that construct arrays usually also include an optional argument to explicitly specify the datatype. Here is an example:

[ ]:
```
x = np.array([1, 2])  # Let numpy choose the datatype
y = np.array([1.0, 2.0])  # Let numpy choose the datatype
z = np.array([1, 2], dtype=np.int64)  # Force a particular datatype
```

```
print(x.dtype, y.dtype, z.dtype)
```

You can read all about numpy datatypes in the documentation.

1.9.4 Array Math

Basic mathematical functions operate elementwise on arrays, and are available both as operator overloads and as functions in the NumPy module:

[ ]:
```
x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)

print(x)
print(y)
```

[ ]:
```
# Elementwise sum; both produce an array
print(x + y)
print(np.add(x, y))
```

[ ]:
```
# Elementwise difference; both produce an array
print(x - y)
print(np.subtract(x, y))
```

[ ]:
```
# Elementwise product; both produce an array
print(x * y)
print(np.multiply(x, y))
```

[ ]:
```
# Elementwise division; both produce an array
print(x / y)
print(np.divide(x, y))
```

[ ]:
```
# Elementwise square root; produces an array
print(np.sqrt(x))
```

Note that, `*` is elementwise multiplication, not matrix multiplication. We instead use the `dot` function to compute inner products of vectors, to multiply a vector by a matrix, and to multiply matrices. `dot` is available both as a function in the numpy module and as an instance method of array objects:

[ ]:
```
v = np.array([9, 10])
w = np.array([11, 12])

# Inner product of vectors; both produce 219
print(v.dot(w))
print(np.dot(v, w))
```

You can also use the `@` operator, which is equivalent to `dot` for 1-D and 2-D arrays.

[ ]:
```
print(v @ w)
```

[ ]:
```
x = np.array([[1,2],[3,4]])
y = np.array([[5,6],[7,8]])

# Matrix / vector product; both produce the 1-D array [29 67]
print(x.dot(v))
print(np.dot(x, v))
print(x @ v)
```

[ ]:
```
# Matrix / matrix product; both produce the 2-D array
print(x.dot(y))
print(np.dot(x, y))
print(x @ y)
```

NumPy provides many useful functions for performing computations on arrays; one of the most useful is `sum`:

[ ]:
```
x = np.array([[1,2],[3,4]])

print(np.sum(x))  # Compute sum of all elements; prints "10"
print(np.sum(x, axis=0))  # Compute sum of each column; prints "[4 6]"
print(np.sum(x, axis=1))  # Compute sum of each row; prints "[3 7]"
```

You can find the full list of mathematical functions provided by numpy in the documentation.

Apart from computing mathematical functions using arrays, we frequently need to reshape or otherwise manipulate data in arrays. The simplest example of this type of operation is transposing a matrix; to transpose a matrix, simply use the T attribute of an array object:

[ ]:
```
x = np.array([[1,2],[3,4]])

print(x)
print("transpose\n", x.T)
```

[ ]:
```
v = np.array([[1,2,3]])
print(v)
print("transpose\n", v.T)
```

1.9.5 Broadcasting

Broadcasting is a powerful mechanism that allows NumPy to work with arrays of different shapes when performing arithmetic operations. Frequently, we have a smaller array and a larger array, and we want to use the smaller array multiple times to perform some operation on the larger array.

For example, suppose that we want to add a constant vector to each row of a matrix. We could do it like this:

[ ]:
```
# We will add the vector v to each row of the matrix x,
# storing the result in the matrix y
x = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
v = np.array([1, 0, 1])
```

```
y = np.empty_like(x)   # Create an empty matrix with the same shape as x

# Add the vector v to each row of the matrix x with an explicit loop
for i in range(4):
    y[i, :] = x[i, :] + v

print(y)
```

This works as we expected. However, when the matrix x is very large, computing an explicit loop in Python could be slow. Note that adding the vector v to each row of the matrix x is equivalent to forming a matrix vv by stacking multiple copies of v vertically, then performing elementwise summation of x and vv. We could implement this approach like this:

[ ]:
```
vv = np.tile(v, (4, 1))  # Stack 4 copies of v on top of each other
print(vv)
```

[ ]:
```
y = x + vv  # Add x and vv elementwise
print(y)
```

NumPy broadcasting allows us to perform this computation without actually creating multiple copies of v. Consider this version, using broadcasting:

[ ]:
```
# We will add the vector v to each row of the matrix x,
# storing the result in the matrix y
x = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
v = np.array([1, 0, 1])
y = x + v  # Add v to each row of x using broadcasting
print(y)
```

The line `y = x + v` works even though `x` has shape `(4, 3)` and `v` has shape `(3,)`, due to **broadcasting**. The `(3,)` array is broadcast across the rows of `x`, so the operation behaves as if `v` had been copied across all four rows and the addition was performed elementwise.

Broadcasting two arrays together follows these rules:

1. If the arrays do not have the same number of dimensions, the shape of the array with fewer dimensions is prepended with 1s until both shapes have the same number of dimensions.
2. The arrays are compatible in a dimension if their sizes are equal, or if the length of one dimension is 1.
3. The arrays can be broadcast together if they are compatible in every dimension.
4. The resulting shape is the elementwise maximum of the two input shapes.
5. In any dimension where one array has size 1 and the other has a larger size, the array with size 1 is broadcast along that dimension.

If this explanation is not clear enough, try reading the explanation from the documentation or this explanation.

Functions that support broadcasting are known as universal functions. You can find the list of all universal functions in the documentation.

Here are some applications of broadcasting:

[ ]:
```
# Add a vector to each column of a matrix
x = np.array([[1,2,3], [4,5,6]])
w = np.array([4,5])

# x has shape (2, 3) and w has shape (2,).
# If we transpose x then it has shape (3, 2) and can be broadcast
# against w to yield a result of shape (3, 2); transposing this result
# yields the final result of shape (2, 3) which is the matrix x with
# the vector w added to each column. Gives the following matrix:
print((x.T + w).T)
```

[ ]:
```
# Another solution is to reshape w to be a column vector of shape (2, 1);
# we can then broadcast it directly against x to produce the same
# output.
print(x + np.reshape(w, (2, 1)))
```

[ ]:
```
# Multiply a matrix by a constant:
# x has shape (2, 3). Numpy treats scalars as arrays of shape ();
# these can be broadcast together to shape (2, 3), producing the
# following array:
print(x * 2)
```

Broadcasting typically makes your code more concise and faster, so you should strive to use it where possible.

This brief overview has touched on many of the important things that you need to know about NumPy. Check out the NumPy reference to find out much more.

Task 5: Define two vectors a = [1,2,3,4] and b = [5,6,7,8], create a matrix X using NumPy in a way that vector a becomes the first row, whereas b becomes the second row of the matrix. Then multiply each value in the matrix by 2.

1.10 Version Control with Git

1.10.1 Why Version Control?

Without version control, collaborative coding can quickly turn into `analysis_final.py`, `analysis_final_v2.py`, and `analysis_final_USE_THIS_ONE.py`. Git solves this by recording snapshots (**commits**) of your project over time. You can see what changed, when it changed, and who made the change; go back to an earlier version if something breaks; and work on the same codebase as five other people without overwriting each other's work.

1.10.2 A Note on GitHub

Most Git tutorials online are written around **GitHub**. GitLab and GitHub are competing services that both host Git repositories, and the Git commands themselves are **identical** on both. What differs is the website, some of the terminology, and how you set up authentication.

GitHub calls it	GitLab calls it
Pull request	Merge request
Collaborator	Member, with a role: Guest, Reporter, Developer, or Maintainer

So, if a tutorial tells you to create a GitHub account and push your code to GitHub, **stop and use your GitLab repository instead**. Everything you learn about staging, committing, and viewing history transfers without change.

1.10.3 Learning Git: Pick a Route

You can use Git either from the **command line** or through the **Source Control panel in VS Code**. Both provide ways to interact with the same Git repository, so the choice is largely a matter of preference. We suggest trying both: the command line makes it clearer what Git is actually doing, while VS Code can be more convenient for everyday work and provides a helpful interface for reviewing changes and resolving merge conflicts.

Start here (20 minutes, no installation needed)

Learn Git Branching is an interactive sandbox that visualises your commits as you work. Working through the first sequence, *Introduction to Git Commits*, is a quick way to build a mental picture of what commits and branches actually are. It is not tied to GitHub or GitLab, so the concepts you learn apply to Git in general.

Route 1: the command line

Software Carpentry: Version Control with Git is a free, well-paced lesson written for researchers. Work through **episodes 1 to 6** (Automated Version Control, Setting Up Git, Creating a Repository, Tracking Changes, Exploring History, and Ignoring Things). These cover the core Git concepts and commands you will need.

Stop after episode 6. The later episodes introduce remote repositories and collaboration using GitHub. Please remember that we will use **GitLab** instead.

Route 2: VS Code

- Version control in VS Code is a short video showing the Git workflow through the Source Control panel.
- Source control in VS Code is the fuller written guide, including the three-way merge editor for resolving conflicts.

Important: VS Code offers a **"Publish to GitHub"** button for local repositories that do not yet have a remote. Do not use this if you are working with GitLab. Your repository will already exist on **GitLab**, so you will **clone** the existing repository rather than create and publish a new one.

1.10.4 Reference

Pro Git and the Atlassian Git tutorials are useful references if you want to look up a particular Git concept or command.

Task 6: Get familiar with GitLab

1. Log in to your account on the university's GitLab instance (https://scc-source.lancs.ac.uk/) and familiarise yourself with the web interface.
2. Create a private remote repository for this week's lab. You will also need to create an access token with sufficient permissions to access the repository from your local machine.
3. Clone the remote repository to your local machine.
4. Add the Python code you have developed so far in this tutorial to your local copy of the repository. Then commit and push your changes to the remote repository.
5. Go to the GitLab web interface and verify that your changes have been pushed successfully.